Demo walkthrough — 8 minutes

AI Food & Calories Recognition

Minute-by-minute walkthrough for mentor / team presentation, 2026-08-30.

0:00–0:45 | Framing

SAY	DO
"We were asked to build a web app where you take a photo of your meal, we identify the food, and tell you the calories — adjustable for portion size. It's a 4-week internship solo project, and we built it to demo grade today."	On screen: show the app landing page. Point out the clean one-column layout: "Upload a photo," big Analyze button. Let it sit for 3 seconds.

0:45–1:30 | The Architecture Decision

SAY	DO
"We had four options: train a model from scratch, fine-tune an existing one, call a paid API, or use a published fine-tune with an open scaffold. We chose the published fine-tune — the `nateraw/food` model on Hugging Face. It's a Vision Transformer trained on 101 food classes, already accurate, so we spent our internship time on the product and UX, not on model training. Speed over reinvention."	Pause 2 seconds. No screen change needed—stay on landing page. The decision is in the narrative, not visual.

1:30–3:00 | Happy Path

SAY	DO
"Let me show you the happy path. I'm going to drag a pizza photo onto the canvas here."	Drag a pizza image file into the upload area (or click to browse and select a pizza photo from `samples/` folder).
"Now we hit Analyze. The model runs in about a second on CPU."	Click the Analyze button. Wait for the spinner to finish (~1–2 seconds).
"Here's the result screen. At the top: the food name — 'Pizza' — with a green confidence badge showing it's high confidence. Below that, a big calorie number: 2,850 kcal for the whole item. Under that, three macro cards: 86g protein, 112g fat, 360g carbs."	Point to each element as it appears: food name, badge color, large calorie display, then the three macro cards. Let the user read for 3 seconds.
"You'll also notice this line here: 'Estimates only — for reference, not medical advice.' That's always visible, because the nutrition data comes from a public database and needs context."	Point to the disclaimer.

3:00–4:00 | Portion Control

SAY	DO
"Now let's adjust for portion. Right now it's showing the nutrition for a full pizza — that's the 'Large' default. If I click 'Small' here, watch the numbers update instantly."	Click the "Small" portion button. Watch the calorie count drop (e.g., to ~1,425 kcal) and macros halve proportionally.
"I can also type in grams directly — 150 grams, say."	Click the grams input field, clear it, type "150". Press Enter or Tab. Watch values recalculate.
"All of this math happens on the client side, in the browser. The app has a nutrition table for each food with per-100g values, so the portion size is instant."	Pause 2 seconds. No screen change.

4:00–5:30 | Low Confidence & Correction

SAY	DO
"Let's test what happens when the model isn't sure. I'm going to upload a more ambiguous photo — a mixed salad, hard to classify."	Drag or upload a salad/ambiguous food image. Click Analyze.
"See the amber badge? That's low confidence. And below the top guess, we show the top-3 alternatives. The model thinks it might be a Caesar salad, Greek salad, or garden salad."	Point to the amber badge and the three alternatives in a dropdown or list.
"If the model is wrong, I can click 'Wrong food?' here, and search for what it actually is. Let me type 'fries'."	Click the "Wrong food?" search field and type "fries".
"Now I pick from the search results — there's regular fries, there's crinkle-cut fries. I'll pick regular fries."	Click on "fries" in the search results. Watch the nutrition update for fries instead.
"See in the sidebar on the left? A history of all the foods I've looked up in this session. If I want to go back and edit the portion for the pizza, I can click it here and it reloads the result."	Point to the session history sidebar. Click one past item to show it reloads.

5:30–6:30 | Measured Accuracy

SAY	DO
"Let's talk accuracy. We tested this model on 100 held-out images from the Food-101 dataset — images it never saw during training. Top-1 accuracy is 87%, meaning the model gets the exact food right 87 times out of 100."	Pause 1 second. No screen change.
"Top-3 accuracy is 97%. That's why the alternatives UI exists — if the model misses the exact food in the top guess, the top-3 almost always includes the right answer. So even on the 13% of cases where we're wrong in first place, 10 of those 13 are still in the alternatives."	Pause 1 second.
"Real example: the model sometimes confuses a Greek salad with a Caesar salad — they look similar. But both are in the top-3, so the user can correct it immediately."	Pause.
"Latency is about 1.1 seconds on CPU. It's not instant, but it's snappy enough for a demo-grade app."	Pause.

6:30–7:15 | Known Limitations

SAY	DO
"A few things this app can't do yet, and we know about them:"	Pause 1 second.
"One, it's single-item only. If you have a plate with pizza and salad on it, the model will pick whichever is most prominent. Multi-item detection is a bigger engineering problem — we left it for a future iteration."	Pause.
"Two, we trained the model on 101 food classes. If you photograph something obscure or outside those 101, the model will guess the closest thing, which might be wrong. The classes are common dishes — pizza, tacos, ramen, salads, proteins, desserts — so most meals should work."	Pause.
"Three, HEIC files — iPhone photos — aren't supported yet. The app needs JPG or PNG. That's a format conversion in the upload step, future work."	Pause.
"Four, the nutrition values are from a public food database. They're flagged with 'estimates only' because real nutrition varies by recipe, ingredient brand, and preparation."	Pause.

7:15–8:00 | Questions & Next Steps

SAY	DO
"Questions?"	Pause 10–15 seconds for questions from the mentor or team.
"If there are none, here are things we'd do in the next iteration: detect multiple items in one photo, estimate portion size from the image itself instead of manual input, and integrate a verified nutrition database so values don't need the disclaimer."	Pause 3 seconds. Stay on screen.
"The app is live on [domain], the code is here [point to GitHub link if visible], and the model is `nateraw/food` on Hugging Face. It's all open source and portable. That's the demo."	Pause. Thank you.

Fallback Plan

If the live backend or model fails during the demo:

1. **Pre-recorded happy path (90 seconds)**
2. Keep a screen recording of the pizza → result → portion adjustment flow on the desktop as `demo-backup.mp4`.
3. If the app is down, open and play it full-screen. Narrate as if it were live.
4. **Accuracy report backup (printed or PDF)**

5. Bring the `accuracy-report.md` file printed or as a PDF on your laptop.
6. If asked for numbers mid-demo, pull up the report and read the tables (87% top-1, 97% top-3, etc.).
7. **Restart command**
8. If the backend is just hung, close the window and run in a terminal: `bash cd <project-root> && python api.py`
9. Wait 10 seconds, then reload the browser. Backend should be live.
10. **Local fallback (no internet)**
11. The model is cached locally after first download.
12. The app frontend is a static bundle (served from `public/`).
13. If there's no internet at the venue, the app still works offline — nothing external is called except the initial model load.
14. The nutrition database is bundled in JSON; no API calls needed.

Pre-Demo Checklist

- [ ] **10 minutes before:** Boot the API server. Run `python api.py` and confirm "Server running on http://localhost:5000" in the terminal.
- [ ] **5 minutes before:** Load the app in the browser. Navigate to `http://localhost:3000` (or the deployed URL). Verify the upload canvas is visible and the Analyze button is clickable.
- [ ] **Test one photo:** Drag a simple image (e.g., pizza) and hit Analyze. Confirm the result screen loads and numbers are visible. Do not demo this test case during the presentation.
- [ ] **Have three sample photos ready on the desktop:**
- One high-confidence photo (e.g., pizza, burger, distinct food)
- One ambiguous photo (e.g., mixed salad, hard to classify)
- One non-food photo (optional, to show error handling — e.g., a shoe, a keyboard)
- [ ] **Close other apps:** Close Slack, email, music, or any background notifications that might pop up during the demo. Full-screen the demo.
- [ ] **Check the venue:** If presenting on a projector, test it works 5 minutes before the demo starts. Confirm WiFi or local-only status.
- [ ] **Have a phone or tablet nearby:** A fallback device to pull up the repo link or slides if needed.